

LAB TASK 13: INTELLIGENT Q&A BOT

Implementation of Semantic Search using FAISS & MiniLM

1. Project Objective

The objective of this task is to evolve the standard keyword-based chatbot into an **Intelligent Q&A Bot**. By implementing a semantic search pipeline, the bot can understand the *context* and *intent* behind a user's query rather than just matching characters. This is achieved using **Vector Embeddings** and **Similarity Search**.

2. Key Concept Definitions

LLMs	Large Language Models (like MiniLM used here) that act as the primary brain to process and understand human language.
Vector	A numerical representation of text. In this project, each style description is converted into a list of numbers.
VectorDB / FAISS	A specialized storage system that allows us to search through vectors based on how "close" they are in meaning.
RAG	Retrieval-Augmented Generation. A process where the bot first "retrieves" the right info from our database before "generating" an answer.

3. The NLP Pipeline

The project follows a standard industry-level AI pipeline:

1. **Preprocessing:** Loading the style Q&A dataset into Python structures.
2. **Embedding:** Using the **Hugging Face 'all-MiniLM-L6-v2'** model to generate numerical vectors for our questions.

3. **Indexing:** Storing these vectors in a **FAISS Index** for high-speed similarity search.
4. **Retrieval:** When a user asks a question, the query is embedded and the most similar vector is fetched using **L2 Distance**.

4. Core Logic

```
# Convert query to vector query_embedding = model.encode([user_query]) # Search
FAISS index for the 1 nearest neighbor D, I =
index.search(np.array(query_embedding), k=1) # Threshold check (Lower distance
= Higher similarity) if D[0][0] < 1.5: response = answers[I[0][0]] else:
response = "Default fallback response."
```

5. Results and Observations

The system was tested with various "informal" and "aesthetic" queries. Unlike Lab 10, the bot successfully identified that a request for **"something clean and simple"** relates to the **Minimalist** style even if the word 'minimalist' was not used. This demonstrates the power of semantic search in real-world AI applications.

Self-Correction: During development, a *ModuleNotFoundError* for FAISS was encountered in the Jupyter Notebook. This was resolved by installing dependencies directly within the notebook kernel using `%pip install faiss-cpu`.